

Core Language Working Group "ready" Issues for the November, 2022 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4917](#).

2392. *new-expression* size check and constant evaluation

Section: 7.7 [[expr.const](#)] **Status:** ready **Submitter:** Tam S. B **Date:** 2018-12-05

According to 7.6.2.8 [[expr.new](#)] paragraph 8, if the *expression* in a *nopt-new-declarator* is a core constant expression, the program is ill-formed if the *expression* is erroneous, e.g., negative. However, consider the following example:

```
template<class T = void> constexpr int f() { T t; return 1; }  
using _ = decltype(new int[f()]);
```

`f()` is a core constant expression, so it must be evaluated to determine its value. However, because the expression appears in an unevaluated operand, it is not “potentially constant evaluated” and thus `f` is not “needed for constant evaluation”, so the template is not instantiated (13.9.2 [[temp.inst](#)] paragraph 7). There is implementation divergence on the handling of this example.

CWG telecon 2022-09-09:

The example should be well-formed, because `f` is not instantiated.

A similar situation arises for narrowing conversions, except that in the latter case, determining the value at compile-time empowers to allow additional cases, whereas the *new-expression* case uses a compile-time value to prohibit additional cases.

Proposed resolution (approved by CWG 2022-09-23):

Change in 7.6.2.8 [[expr.new](#)] paragraph 8 as follows:

If the expression is erroneous after converting to `std::size_t`:

- if the expression is a **potentially-evaluated** core constant expression, the program is ill-formed;
- otherwise, an allocation function is not called; instead...

2407. Missing entry in Annex C for defaulted comparison operators

Section: Clause Annex C [[diff](#)] **Status:** ready **Submitter:** Tomasz Kaminski **Date:** 2019-02-26

The changes from [P1185R2](#) need an entry in Annex C, because they affect the interpretation of existing well-formed code. For example, given:

```
struct A {  
    operator int() const { return 10; }  
};  
  
bool operator==(A, int); // #1  
//built-in: bool operator==(int, int); // #2  
  
A a, b;
```

The expression `10 == a` resolves to #2 in C++17 but now to #1. In addition, `a == b` is now ambiguous, because #1 has a user-defined conversion on the second argument, while the reversed order has it on the first argument. Similarly for `operator!=`.

Notes from the March, 2019 teleconference:

The ambiguity in `10 == a` arises from the consideration of the reverse ordering of the operands.

CWG found this breakage surprising and asked for EWG's opinion before updating Annex C.

Proposed resolution (April, 2019) [SUPERSEDED]

Add the following as a new subclause in C.2 [diff.cpp17]:

C.5.6 Clause 12: Overloading

Affected subclause: 12.2.2.3 [over.match.oper]

Change: Overload resolution may change for equality operators 7.6.10 [expr.eq].

Rationale: Support calling `operator==` with reversed order of arguments.

Effect on original feature: Valid C++ 2017 code that uses equality operators with conversion functions may be ill-formed or have different semantics in this International Standard.

```
struct A {
    operator int() const { return 10; }
};
```

```
bool operator==(A, int);           // #1
```

```
// built-in: bool operator==(int, int); // #2
```

```
bool b = 10 == A();               // uses #1 with reversed order of arguments; previously used #2
```

Proposed resolution:

Add the following as a new subclause in C.2 [diff.cpp17]:

C.5.6 Clause 12: Overloading

Affected subclause: 12.2.2.3 [over.match.oper]

Change: Overload resolution may change for equality operators 7.6.10 [expr.eq].

Rationale: Support calling `operator==` with reversed order of arguments.

Effect on original feature: Valid C++ 2017 code that uses equality operators with conversion functions may be ill-formed or have different semantics in this International Standard.

```
struct A {
    operator int() const { return 10; }
};
```

```
bool operator==(A, int);           // #1
```

```
// built-in: bool operator==(int, int); // #2
```

```
bool b = 10 == A();               // uses #1 with reversed order of arguments; previously used #2
```

```
struct B {
    bool operator==(const B&);      // member function with no cv-qualifier
};
B b1;
bool eq = (b1 == b1);              // ambiguous; previously well-formed
```

2410. Implicit calls of immediate functions

Section: 9.2.6 [dcl.constexpr] **Status:** ready **Submitter:** John Spicer **Date:** 2019-03-27

The intent for immediate functions is that they can only be called at compile time. That rule is enforced by the wording of 7.5.4 [expr.prim.id] paragraph 3:

An *id-expression* that denotes an immediate function (9.2.6 [dcl.constexpr]) shall appear as a subexpression of an immediate invocation or in an immediate function context (7.7 [expr.const]).

However, this restriction does not apply to implicit function calls such as constructor and operator invocations. Presumably some additional wording is needed for such cases.

Additional note, July, 2019:

This issue would appear to be NAD because of the following wording from 7.7 [expr.const] paragraph 10:

An expression or conversion is an immediate invocation if it is an explicit or implicit invocation of an immediate function and is not in an immediate function context. An immediate invocation shall be a constant expression.

Proposed resolution (approved by CWG 2022-10-21):

Change in 7.7 [expr.const] paragraph 10 as follows:

An ~~expression or conversion~~ **invocation** is an immediate invocation if it is an explicit or implicit invocation of an immediate function and is not in an immediate function context. An immediate invocation shall be a constant expression.

2428. Deprecating a concept

Section: 13.7.9 [temp.concept] **Status:** ready **Submitter:** Eric Niebler **Date:** 2018-12-10

The grammar for a *concept-definition* does not include an *attribute-specifier-seq_{opt}*, making it impossible to deprecate an attribute. This seems like an oversight.

CWG telecon 2022-10-07:

Agreed.

Proposed resolution (approved by CWG 2022-10-21):

1. Change in 9.12.5 [dcl.attr.deprecated] paragraph 2 as follows:

The attribute may be applied to the declaration of a class, a *typedef-name*, a variable, a non-static data member, a function, a namespace, an enumeration, an enumerator, **a concept**, or a template specialization.

2. Change in 13.7.9 [temp.concept] paragraph 1 as follows:

A concept is a template that defines constraints on its template arguments.

concept-definition:

concept *concept-name* **attribute-specifier-seq_{opt}** = *constraint-expression* ;

concept-name:

identifier

A *concept-definition* declares a concept. Its *identifier* becomes a *concept-name* referring to that concept within its scope. **The optional attribute-specifier-seq appertains to the concept.**

2440. Allocation in core constant expressions

Section: 7.7 [expr.const] **Status:** ready **Submitter:** Davis Herring **Date:** 2019-08-28

7.7 [expr.const] paragraph 5 attempts to describe allowable allocation/deallocation calls in terms of what could be called “core constant subexpressions,” but the actual definition of a core constant expression in paragraph 4 is in terms of evaluation.

Suggested resolution [SUPERSEDED]:

Replace the entirety of 7.7 [expr.const] paragraph 6 with the following:

For the purposes of determining whether an expression *E* is a core constant expression, the evaluation of a call to the **body of** a member function of `std::allocator<T>` as defined in 20.2.10.2 [allocator.members], where *T* is a literal type, does not disqualify *E* from being a core constant expression, even if the actual evaluation of such a call would otherwise fail the requirements for a core constant expression **is ignored**. Similarly, the evaluation of a call to the **body of** `std::destroy_at`, `std::ranges::destroy_at`, `std::construct_at`, or `std::ranges::construct_at` (27.11.8 [specialized.construct]) does not disqualify *E* from being a core constant expression unless the first argument, of type *T**, does not point to storage allocated with `std::allocator<T>` or to an object whose lifetime began within the evaluation of *E*, or the evaluation of **is considered to include only** the underlying constructor call disqualifies *E* from being a core constant expression **(for the functions `construct_at`) or destructor (for the functions `destroy_at`) call if the first argument (of type *T**) points to storage allocated with `std::allocator<T>`.**

CWG telecon 2022-10-21:

The references to `destroy_at` were removed in an unrelated update to the Working Draft. Also, restore the reference to local objects whose lifetime began within *E*.

Proposed resolution (approved by CWG 2022-11-10):

Change in 7.7 [expr.const] paragraph 6 as follows:

For the purposes of determining whether an expression *E* is a core constant expression, the evaluation of a call to the **body of** a member function of `std::allocator<T>` as defined in 20.2.10.2 [allocator.members], where *T* is a literal type, does not disqualify *E* from being a core constant expression, even if the actual evaluation of such a call would otherwise fail the requirements for a core constant expression **is ignored**. Similarly, the evaluation of a call to the **body of** `std::construct_at` or `std::ranges::construct_at` (27.11.8 [specialized.construct]) does not disqualify *E* from being a core constant expression unless the first argument, of type *T**, does not point to storage allocated with `std::allocator<T>` or to an object whose lifetime began within the evaluation of *E*, or the evaluation of **is considered to include only** the underlying constructor call disqualifies *E* from being a core constant expression **call if the first argument (of type *T**) points to storage allocated with `std::allocator<T>` or to an object whose lifetime began within the evaluation of *E*.**

2451. *promise.unhandled_exception()* and final suspend point

Section: 9.5.4 [dcl.fct.def.coroutine] **Status:** ready **Submitter:** Lewis Baker **Date:** 2020-02-14

According to 9.5.4 [dcl.fct.def.coroutine] paragraph 14,

If the evaluation of the expression *promise.unhandled_exception()* exits via an exception, the coroutine is considered suspended at the final suspend point.

However, the “final suspend point” is defined as being “the *await-expression* containing the call to `final_suspend`” (bullet 5.2), and it is not desired to evaluate the `final_suspend` expression in this case.

Suggested resolution [SUPERSEDED]:

1. Change 9.5.4 [dcl.fct.def.coroutine] paragraph 5 as follows:

...where

- the *await-expression* containing the call to `initial_suspend` is the *initial suspend point await expression*, and
- the *await-expression* containing the call to `final_suspend` is the *final suspend point await expression*, and

- *initial-await-resume-called* is initially `false` and is set to `true` immediately before the evaluation of the *await-resume* expression (7.6.2.4 [expr.await]) of the initial ~~suspend point~~ **await expression**, and
- ...
- *promise-constructor-arguments* is determined as follows: overload resolution is performed on a promise constructor call created by assembling an argument list with lvalues $p_1 \dots p_n$. If a viable constructor is found (12.2.3 [over.match.viable]), then *promise-constructor-arguments* is $(p_1, \dots, p_n)$, otherwise *promise-constructor-arguments* is empty; **, and**
- **A coroutine is suspended at a final suspend point if it is suspended**
 - **at a final await expression or**
 - **due to an exception exiting from `unhandled_exception()`**

2. Change bullet 3.2 of 7.6.2.4 [expr.await] as follows:

Evaluation of an *await-expression* involves the following auxiliary types, expressions, and objects:

- ...
- *a* is the *cast-expression* if the *await-expression* was implicitly produced by a *yield-expression* (7.6.17 [expr.yield]), an initial ~~suspend point~~ **await expression**, or a final ~~suspend point~~ **await expression** (9.5.4 [dcl.fct.def.coroutine]). Otherwise
- ...

3. If needed, change 9.5.4 [dcl.fct.def.coroutine] paragraph 14 as follows:

If the evaluation of the expression `promise.unhandled_exception()` exits via an exception, the coroutine is considered suspended at the final suspend point **and the exception propagates to the caller or resumer.**

Notes from the August, 2020 teleconference [SUPERSEDED]:

CWG expressed some concern about the lack of a precise definition of “suspend point”. Gor Nishanov suggests the following change, in 7.6.2.4 [expr.await] bullet 5.1:

- If the evaluation of *await-suspend* exits via an exception, the exception is caught, the coroutine is resumed, and the exception is immediately re-thrown (14.2 [except.throw]). Otherwise, control flow returns to the current coroutine caller or resumer (9.5.4 [dcl.fct.def.coroutine]) without exiting any scopes (8.7 [stmt.jump]). **The point in the coroutine immediately prior to control returning to its caller or resumer is a coroutine suspend point.**

Proposed resolution (approved by CWG 2022-11-10):

1. Change in 7.6.2.4 [expr.await] bullet 3.2 as follows:

Evaluation of an *await-expression* involves the following auxiliary types, expressions, and objects:

- ...
- *a* is the *cast-expression* if the *await-expression* was implicitly produced by a *yield-expression* (7.6.17 [expr.yield]), an initial ~~suspend point~~ **await expression**, or a final ~~suspend point~~ **await expression** (9.5.4 [dcl.fct.def.coroutine]). Otherwise
- ...

2. Change in 7.6.2.4 [expr.await] bullet 5.1 as follows:

- If the evaluation of *await-suspend* exits via an exception, the exception is caught, the coroutine is resumed, and the exception is immediately re-thrown (14.2 [except.throw]). Otherwise, control flow returns to the current coroutine caller or resumer (9.5.4 [dcl.fct.def.coroutine]) without exiting any scopes (8.7 [stmt.jump]). **The point in the coroutine immediately prior to control returning to its caller or resumer is a coroutine suspend point.**

3. Change in 9.5.4 [[dcl.fct.def.coroutine](#)] paragraph 5 as follows:

...where

- the *await-expression* containing the call to `initial_suspend` is the *initial suspend point await expression*, and
- the *await-expression* containing the call to `final_suspend` is the *final suspend point await expression*, and
- *initial-await-resume-called* is initially `false` and is set to `true` immediately before the evaluation of the *await-resume* expression (7.6.2.4 [[expr.await](#)]) of the *initial suspend point await expression*, and
- ...
- *promise-constructor-arguments* is determined as follows: overload resolution is performed on a promise constructor call created by assembling an argument list with lvalues $p_1 \dots p_n$. If a viable constructor is found (12.2.3 [[over.match.viable](#)]), then *promise-constructor-arguments* is $(p_1, \dots, p_n)$, otherwise *promise-constructor-arguments* is empty, and
- a coroutine is suspended at the *initial suspend point* if it is suspended at the *initial await expression*, and
- a coroutine is suspended at a *final suspend point* if it is suspended
 - at a final await expression or
 - due to an exception exiting from `unhandled_exception()`

4. Change 9.5.4 [[dcl.fct.def.coroutine](#)] paragraph 14 as follows:

If the evaluation of the expression `promise.unhandled_exception()` exits via an exception, the coroutine is considered suspended at the final suspend point and the exception propagates to the caller or resumer.

2508. Restrictions on uses of template parameter names

Section: 13.8.2 [[temp.local](#)] **Status:** ready **Submitter:** Daveed Vandevoorde **Date:** 2021-11-01

The status of an example like the following is unclear:

```
template<typename T> T T(T) {}
```

According to 13.8.2 [[temp.local](#)] paragraph 6,

The name of a *template-parameter* shall not be bound to any following declaration contained by the scope to which the *template-parameter* belongs. [Example 5:

```
...
template<class X> class X; // error: hidden by template-parameter
```

—end example]

The intent would appear to be that the function template could not have the same name as the template parameter. However, according to 6.4.9 [[basic.scope.temp](#)] paragraph 2,

Each *template-declaration* D introduces a template parameter scope that extends from the beginning of its *template-parameter-list* to the end of the *template-declaration*. Any declaration outside the *template-parameter-list* that would inhabit that scope instead inhabits the same scope as D .

This would indicate that the function template inhabits the namespace scope, not the template parameter scope, so the prohibition against use of the template parameter name would not apply.

To reject both the function and class template examples, 13.8.2 [temp.local] paragraph 6 could be changed to read:

The name of a *template-parameter* shall not be bound to any following declaration **whose locus is** contained by the scope to which the *template-parameter* belongs.

To accept both examples, the change could be:

The name of a *template-parameter* shall not be bound to any following declaration **that inhabits a scope** contained by the scope to which the *template-parameter* belongs.

Notes from the December, 2021 teleconference:

The consensus of CWG was to reject both examples, i.e., the first option.

Additional note (December, 2021):

It was observed that this issue is, strictly speaking, not a defect: the word “contains” is used in 6.4.1 [basic.scope.scope] paragraph 1 in its usual English sense to refer to the lexical nesting of scopes, so the template parameter scope of τ “contains” the declaration of the function τ . However, the use of the term “locus” would make the intent clearer.

Proposed resolution (December, 2021):

Change 13.8.2 [temp.local] paragraph 6 as follows:

The name of a *template-parameter* shall not be bound to any following declaration **whose locus is** contained by the scope to which the *template-parameter* belongs.

2583. Common initial sequence should consider over-alignment

Section: 11.4.1 [class.mem.general] **Status:** ready **Submitter:** Brian Bi **Date:** 2022-05-03

Consider:

```
struct A {
    int i;
    char c;
};

struct B {
    int i;
    alignas(8) char c;
};

union U { A a; B b; };
```

On a lot of platforms, A and B do not have the same layout, yet 11.4.1 [class.mem.general] paragraph 23 does not consider differences in alignment in the rules for "common initial sequence":

The common initial sequence of two standard-layout struct (11.2 [class.prop]) types is the longest sequence of non-static data members and bit-fields in declaration order, starting with the first such entity in each of the structs, such that corresponding entities have layout-compatible types (6.8 [basic.types]), either both entities are declared with the `no_unique_address` attribute (9.12.11 [dcl.attr.nouniqueaddr]) or neither is, and either both entities are bit-fields with the same width or neither is a bit-field.

In the following example,

```
struct S0 {
    alignas(16) char x[128];
    int i;
};
struct alignas(16) S1 {
    char x[128];
    int i;
};
```

S0 and S1 have the same alignment, yet per the suggested rules below, they will not be layout-compatible.

Suggested resolution [SUPERSEDED]:

Change in 11.4.1 [class.mem.general] paragraphs 23-25 as follows (also add bullets):

The common initial sequence of two standard-layout struct (11.2 [class.prop]) types is the longest sequence of non-static data members and bit-fields in declaration order, starting with the first such entity in each of the structs, such that

- corresponding entities have layout-compatible types (6.8 [basic.types]),
- **either both entities have alignment-specifiers that specify equivalent alignment or neither entity has an alignment-specifier (9.12.2 [dcl.align]),**
- either both entities are declared with the no_unique_address attribute (9.12.11 [dcl.attr.nouniqueaddr]) or neither is, and
- either both entities are bit-fields with the same width or neither is a bit-field.

[...]

Two standard-layout struct (11.2 [class.prop]) types are layout-compatible classes if their common initial sequence comprises all members and bit-fields of both classes (6.8 [basic.types]) **and either both types are declared with alignment-specifiers that specify equivalent alignment or neither type has an alignment-specifier.**

Two standard-layout unions are layout-compatible if they have the same number of non-static data members and corresponding non-static data members (in any order)

- have layout-compatible types (6.8.1 [basic.types.general]) **and**
- **either both have alignment-specifiers that specify equivalent alignment or neither has an alignment-specifier.**

Proposed resolution (approved by CWG telecon 2022-08-26):

Change in 11.4.1 [class.mem.general] paragraphs 23-25 as follows (also add bullets):

The common initial sequence of two standard-layout struct (11.2 [class.prop]) types is the longest sequence of non-static data members and bit-fields in declaration order, starting with the first such entity in each of the structs, such that

- corresponding entities have layout-compatible types (6.8 [basic.types]),
- **corresponding entities have the same alignment requirements (6.7.6 [basic.align]),**
- either both entities are declared with the no_unique_address attribute (9.12.11 [dcl.attr.nouniqueaddr]) or neither is, and
- either both entities are bit-fields with the same width or neither is a bit-field.

[...]

2590. Underlying type should determine size and alignment requirements of an enum

Section: 9.7.1 [dcl.enum] **Status:** ready **Submitter:** Brian Bi **Date:** 2022-05-15

Subclause 9.7.1 [dcl.enum] specifies how the underlying type of an enumeration is determined, and, for enumerations whose underlying type is fixed, specifies that the enumeration has the same set of values as the underlying type. However, the specification does not relate the size and alignment requirements of the enumeration to those of the underlying type. Those ought to be the same.

Suggested resolution [SUPERSEDED]:

Add a new paragraph after 9.7.1 [dcl.enum] paragraph 8:

For an enumeration whose underlying type is fixed, ...

An enumeration has the same size, value representation, and alignment requirements (6.7.6 [basic.align]) as its underlying type. Furthermore, each value of an enumeration has the same representation as the same value of the underlying type.

Two enumeration types are *layout-compatible enumerations* if ...

Proposed resolution (approved by CWG 2022-08-26):

Add a new paragraph after 9.7.1 [dcl.enum] paragraph 8:

For an enumeration whose underlying type is fixed, ...

An enumeration has the same size, value representation, and alignment requirements (6.7.6 [basic.align]) as its underlying type. Furthermore, each value of an enumeration has the same representation as the corresponding value of the underlying type.

Two enumeration types are *layout-compatible enumerations* if ...

2598. Unions should not require a non-static data member of literal type

Section: 6.8.1 [basic.types.general] **Status:** ready **Submitter:** Richard Smith **Date:** 2022-06-18

According to 6.8.1 [basic.types.general] paragraph 10, a type is a literal type only if it satisfies the following:

A type is a *literal type* if it is:

- ...
- a possibly cv-qualified class type (Clause 11 [class]) that has all of the following properties:
 - it has a constexpr destructor (9.2.6 [dcl.constexpr]),
 - it is either a closure type (7.5.5.2 [expr.prim.lambda.closure]), an aggregate type (9.4.2 [dcl.init.aggr]), or has at least one constexpr constructor or constructor template (possibly inherited (9.9 [namespace.udecl]) from a base class) that is not a copy or move constructor,
 - if it is a union, at least one of its non-static data members is of non-volatile literal type, and
 - if it is not a union, all of its non-static data members and base classes are of non-volatile literal types.

[Note 4: A literal type is one for which it might be possible to create an object within a constant expression. ... —end note]

However, the normative rule disagrees with the note. Consider:

```
struct A { A(); };
union U {
    A a;
    constexpr U() {}
    constexpr ~U() {}
};
```

It is certainly possible to create an object of type U in a constant expression, even though U is not a literal type.

In the suggested resolution, the aggregate type rule is intended to capture the fact that it is not possible for aggregate initialization of a non-empty union to leave no active member (and similarly for each anonymous union member in a non-union union-like class).

Suggested resolution [SUPERSEDED]:

Change in 6.8.1 [basic.types.general] paragraph 10 as follows:

A type is a *literal type* if it is:

- ...
- a possibly cv-qualified class type (Clause 11 [class]) that has all of the following properties:
 - it has a constexpr destructor (9.2.6 [dcl.constexpr]),

- it ~~is~~ either
 - **is** a closure type (7.5.5.2 [[expr.prim.lambda.closure](#)]),
 - **is** an aggregate type (9.4.2 [[dcl.init.aggr](#)]) **for which that type (if it is a union) or each of its anonymous union members (otherwise) either has at least one variant member of non-volatile literal type or has no variant members,** or
 - has at least one constexpr constructor or constructor template (possibly inherited (9.9 [[namespace.udecl](#)]) from a base class) that is not a copy or move constructor, **and**
- ~~if it is a union, at least one of its non-static data members is of non-volatile literal type, and~~
- ~~if it is not a union,~~ all of its non-static **non-variant** data members and base classes are of non-volatile literal types.

Proposed resolution (CWG telecon 2022-08-12) [SUPERSEDED]:

Change in 6.8.1 [[basic.types.general](#)] paragraph 10 as follows:

A type is a *literal type* if it is:

- ...
- a possibly cv-qualified class type (Clause 11 [[class](#)]) that has all of the following properties:
 - it has a constexpr destructor (9.2.6 [[dcl.constexpr](#)]),
 - **all of its non-static non-variant data members and base classes are of non-volatile literal types, and**
 - it ~~is~~ either
 - **is** a closure type (7.5.5.2 [[expr.prim.lambda.closure](#)]),
 - **is** an aggregate type (9.4.2 [[dcl.init.aggr](#)]) **for which that type (if it is a union) or each of its anonymous union members (otherwise) either has at least one variant member of non-volatile literal type or has no variant members,** or
 - has at least one constexpr constructor or constructor template (possibly inherited (9.9 [[namespace.udecl](#)]) from a base class) that is not a copy or move constructor; **and**
 - ~~if it is a union, at least one of its non-static data members is of non-volatile literal type, and~~
 - ~~if it is not a union, all of its non-static data members and base classes are of non-volatile literal types.~~

Proposed resolution (approved by CWG 2022-11-09):

Change 6.8.1 [[basic.types.general](#)] paragraph 10 as follows:

A type is a *literal type* if it is:

- ...
- a possibly cv-qualified class type (Clause 11 [[class](#)]) that has all of the following properties:
 - it has a constexpr destructor (9.2.6 [[dcl.constexpr](#)]),
 - **all of its non-static non-variant data members and base classes are of non-volatile literal types, and**
 - it ~~is~~ either
 - **is** a closure type (7.5.5.2 [[expr.prim.lambda.closure](#)]),
 - ~~an aggregate type (9.4.2 [[dcl.init.aggr](#)]),~~
 - **is an aggregate union type that has either no variant members or at least one variant member of non-volatile literal type,**
 - **is a non-union aggregate type for which each of its anonymous union members satisfies the above requirements for an aggregate union type, or**
 - has at least one constexpr constructor or constructor template (possibly inherited (9.9 [[namespace.udecl](#)]) from a base class) that is not a copy or move constructor; **and**
 - ~~if it is a union, at least one of its non-static data members is of non-volatile literal type, and~~
 - ~~if it is not a union, all of its non-static data members and base classes are of non-volatile literal types.~~

2599. What does initializing a parameter include?

Section: 7.6.1.3 [[expr.call](#)] **Status:** ready **Submitter:** Davis Herring **Date:** 2022-06-18

Subclause 7.6.1.3 [[expr.call](#)] paragraph 8 specifies:

The *postfix-expression* is sequenced before each expression in the *expression-list* and any default argument. The initialization of a parameter, including every associated value computation and side effect, is indeterminately sequenced with respect to that of any other parameter. [Note 8: All side effects of argument evaluations are sequenced before the function is entered (see 6.9.1 [[intro.execution](#)]). —end note]

Consider:

```
f(std::unique_ptr<int>(new int), std::unique_ptr<int>(new int));
```

It is not clear from the phrasing whether the evaluation of each `new int` is part of the "initialization of [its] parameter" or whether only the initialization of `f`'s parameters from the completed `std::unique_ptr<int>` objects is included. The note does not help, since it can be read as distinguishing argument evaluations from initialization.

Suggested resolution [[SUPERSEDED](#)]:

Insert before 9.4.1 [[dcl.init.general](#)] paragraph 18 as follows:

An *initializer-clause* followed by an ellipsis is a pack expansion (13.7.4 [[temp.variadic](#)]).

Initialization includes the evaluation of all subexpressions of each *initializer-clause* of the initializer (possibly nested within *braced-init-lists*).

If the initializer is a parenthesized *expression-list*, the expressions are evaluated in the order specified for function calls (7.6.1.3 [[expr.call](#)]).

Proposed resolution (approved by CWG 2022-08-26):

Insert before 9.4.1 [[dcl.init.general](#)] paragraph 18 as follows:

An *initializer-clause* followed by an ellipsis is a pack expansion (13.7.4 [[temp.variadic](#)]).

Initialization includes the evaluation of all subexpressions of each *initializer-clause* of the initializer (possibly nested within *braced-init-lists*) and the creation of any temporary objects for function arguments or return values (6.7.7 [[class.temporary](#)]).

If the initializer is a parenthesized *expression-list*, the expressions are evaluated in the order specified for function calls (7.6.1.3 [[expr.call](#)]).

2601. Tracking of created and destroyed subobjects

Section: 14.3 [[except.ctor](#)] **Status:** ready **Submitter:** Richard Smith **Date:** 2022-06-16

Subclause 14.3 [[except.ctor](#)] paragraph 3 specifies:

If the initialization or destruction of an object other than by delegating constructor is terminated by an exception, the destructor is invoked for each of the object's direct subobjects and, for a complete object, virtual base class subobjects, whose initialization has completed (9.4 [[dcl.init](#)]) and whose destructor has not yet begun execution, except that in the case of destruction, the variant members of a union-like class are not destroyed.

A traditional implementation has no way of knowing which subobjects are in the state that their initialization has completed but their destructor has not yet begun execution. For example, the program might call the destructor explicitly, and the implementation

does not track whether that has happened. The intent here is that it only matters whether the implied destructor call generated implicitly as part of the class object's destructor has begun yet, but it does not say that, and the reference to variant members reinforces the interpretation that the set of subobjects that are destroyed is determined dynamically based on which objects are within their lifetimes. Also, combining construction and destruction rules here confuses the matter further -- in "whose initialization has completed and whose destructor has not yet begun" we care exclusively about the first part in constructors and exclusively about the second part in destructors.

The set of things that we actually want to destroy here is the things that were initialized by the initialization (constructor or aggregate initializer) itself, not the things that have been constructed and not destroyed by evaluations that the initialization happens to perform. For example:

```
struct A {
    union { T x; U y; };
    A() { throw "does not destroy x"; }
    A(int) : x() { throw "does destroy x"; }
    A(float) : x() { x.~T(); throw "still destroys x, oops"; }
    A(double) : x() {
        x.~T();
        new(&y) U();
        throw "destroys x, does not destroy y";
    }
};
```

and similarly for aggregate initialization:

```
struct B {
    union { T x; U y; };
    int a;
};
B b = { .x = {}, .a = (b.x.~T(), new (&b.y) U(), throw "destroys x not y")};
```

Destruction is completely different: we just want to destroy all the things that the destructor was going to destroy anyway and hasn't already started destroying.

Suggested resolution [SUPERSEDED]:

Change in 14.3 [except.ctor] paragraph 3 as follows:

If the initialization or destruction of an object other than by delegating constructor is terminated by an exception, the destructor is invoked for each of the object's direct subobjects and, for a complete object, virtual base class subobjects that were directly initialized by the object's initialization and, whose initialization has completed (9.4 [dcl.init]) and whose destructor has not yet begun execution, except that in the case of destruction, the variant members of a union-like class are not destroyed. A subobject is directly initialized if its initialization is specified in 11.9.3 [class.base.init] for initialization by constructor, in 11.9.4 [class.inhctor.init] for initialization by inherited constructor, in 9.4.2 [dcl.init.aggr] for aggregate initialization, or in 9.4.1 [dcl.init.general] for default-initialization, value-initialization, or direct-initialization of an array. [Note: This includes virtual base class subobjects if the initialization is for a complete object, and can include variant members that were nominated explicitly by a mem-initializer or designated-initializer-clause or that have a default member initializer. -- end note]

If the destructor of an object is terminated by an exception, each destructor invocation that would be performed after executing the body of the destructor (11.4.7 [class.dtor]) and that has not yet begun execution is performed. [Note: This includes virtual base class subobjects if the destructor was invoked for a complete object. -- end note]

Proposed resolution (CWG telecon 2022-08-12) [SUPERSEDED]:

Change in 14.3 [except.ctor] paragraph 3 as follows:

If the initialization or destruction of an object other than by delegating constructor is terminated by an exception, the destructor is invoked for each of the object's direct subobjects and, for a complete object, virtual base class subobjects that were known to be initialized by the object's initialization and, whose initialization has completed (9.4 [dcl.init]) and whose destructor has not yet begun execution, except that in the case of destruction, the variant members of a union-like class are not destroyed. A subobject is known to be initialized if its initialization is specified in 11.9.3 [class.base.init] for initialization by constructor, in 11.9.4 [class.inhctor.init] for initialization by inherited constructor, in 9.4.2 [dcl.init.aggr] for aggregate initialization, or in 9.4.1 [dcl.init.general] for

default-initialization, value-initialization, or direct-initialization of an array. [Note: This includes virtual base class subobjects if the initialization is for a complete object, and can include variant members that were nominated explicitly by a *mem-initializer* or *designated-initializer-clause* or that have a default member initializer. -- end note]

If the destructor of an object is terminated by an exception, each destructor invocation that would be performed after executing the body of the destructor (11.4.7 [class.dtor]) and that has not yet begun execution is performed. [Note: This includes virtual base class subobjects if the destructor was invoked for a complete object. -- end note]

Additional notes (August, 2022):

The proposed resolution above does not handle the situation where the initialization of a closure object is terminated by an exception during the evaluation of a lambda expression. It also does not handle 11.4.5.3 [class.copy.ctor] bullet 14.1 (array copies in defaulted constructors).

Proposed resolution (approved by CWG telecon 2022-09-09):

Change in 14.3 [except.ctor] paragraph 3 as follows:

If the initialization or destruction of an object other than by delegating constructor is terminated by an exception, the destructor is invoked for each of the object's ~~direct subobjects and, for a complete object, virtual base class subobjects that were known to be initialized by the object's initialization and~~ ; whose initialization has completed (9.4 [dcl.init]) ~~and whose destructor has not yet begun execution, except that in the case of destruction, the variant members of a union-like class are not destroyed.~~ [Note: If such an object has a reference member that extends the lifetime of a temporary object, this ends the lifetime of the reference member, so the lifetime of the temporary object is effectively not extended. —end note] **A subobject is known to be initialized if its initialization is specified**

- in 11.9.3 [class.base.init] for initialization by constructor,
- in 11.4.5.3 [class.copy.ctor] for initialization by defaulted copy/move constructor,
- in 11.9.4 [class.inhctor.init] for initialization by inherited constructor,
- in 9.4.2 [dcl.init.aggr] for aggregate initialization,
- in 7.5.5.3 [expr.prim.lambda.capture] for the initialization of the closure object when evaluating a *lambda-expression*,
- in 9.4.1 [dcl.init.general] for default-initialization, value-initialization, or direct-initialization of an array.

[Note: This includes virtual base class subobjects if the initialization is for a complete object, and can include variant members that were nominated explicitly by a *mem-initializer* or *designated-initializer-clause* or that have a default member initializer. —end note]

If the destructor of an object is terminated by an exception, each destructor invocation that would be performed after executing the body of the destructor (11.4.7 [class.dtor]) and that has not yet begun execution is performed. [Note: This includes virtual base class subobjects if the destructor was invoked for a complete object. —end note]

The subobjects are destroyed in the reverse order of the completion of their construction. Such destruction is sequenced before entering a handler of the *function-try-block* of the constructor or destructor, if any.

2602. consteval defaulted functions

Section: 9.2.6 [dcl.constexpr] **Status:** ready **Submitter:** Aaron Ballman **Date:** 2022-06-16

It is not clear whether a defaulted consteval function is still an immediate function even if it is not a valid constexpr function. For example:

```
template <typename Ty>
struct A {
    Ty n;
    consteval A() {}
};
```

```

template <typename Ty>
struct B {
    Ty n;
    consteval B() = default;
};

A<int> a;
B<int> b;

```

The declarations of `a` and `b` should both fail due to an uninitialized member `n` in each of `A` and `B`. The `= default;` should not make a difference. However, there is implementation divergence. We should be able to lean on 7.7 [expr.const] bullet 5.5 to handle this when the immediate invocation is required.

Possible resolution:

Change in 9.2.6 [dcl.constexpr] paragraph 7 as follows:

If the instantiated template specialization of a constexpr **templated** function ~~template or member function of a class template~~ would fail to satisfy the requirements for a constexpr function, that specialization is still a constexpr function, even though a call to such a function cannot appear in a constant expression. **Similarly, if the instantiated template specialization of a consteval templated function would fail to satisfy the requirements for a consteval function, that specialization is still an immediate function, even though an immediate invocation would be ill-formed.** If no specialization of the template would satisfy the requirements for a constexpr **or consteval** function when considered as a non-template function, the template is ill-formed, no diagnostic required.

Proposed resolution (August, 2022):

Change in 9.2.6 [dcl.constexpr] paragraph 4 as follows:

If the instantiated template specialization of a constexpr **templated** function ~~template or member function of a class template~~ would fail to satisfy the requirements for a constexpr function, that specialization is still a constexpr function, even though a call to such a function cannot appear in a constant expression. **Similarly, if the instantiated template specialization of a consteval templated function would fail to satisfy the requirements for a consteval function, that specialization is still an immediate function, even though an immediate invocation would be ill-formed.**

2603. Holistic functional equivalence for function templates

Section: 13.7.7.2 [temp.over.link] **Status:** ready **Submitter:** Davis Herring **Date:** 2022-06-20

In C++20, 13.7.7.2 [temp.over.link] paragraph 7 defined equivalence for function templates in terms of equivalence of several of its components; functional equivalence for them was similar in that it was defined recursively for their "return types and parameter lists", but differed with regard to constraints in that it required that they "accept and are satisfied by the same set of template argument lists". P1787R6 simplified the treatment by relying entirely on the "depends on whether two constructs are equivalent, and they are functionally equivalent but not equivalent" rule to make the correspondence check between the function templates ill-formed, no diagnostic required.

This created a situation where moving a constraint between a *template-head* and a *requires-clause* makes a function template truly different (because there is no reasonable way to read 6.4.1 [basic.scope.scope] bullet 4.3.2's "equivalent [...], *template-heads*, and *trailing requires-clauses* (if any)" as requiring a *joint* check for functional equivalence), even if overload resolution would never be able to distinguish them.

Suggested resolution [SUPERSEDED]:

Change in 13.7.7.2 [temp.over.link] paragraph 7 as follows:

If the validity or meaning of the program depends on whether two constructs are equivalent, and they are functionally equivalent but not equivalent, the program is ill-formed, no diagnostic required. **Furthermore, if two function templates do not correspond, but accept and are satisfied by the same set of template argument lists, the program is ill-formed, no diagnostic required.**

1. Append to 6.4.1 [basic.scope.scope] paragraph 3 as follows:

- ...
- the types of their object parameters are equivalent.

Two function templates have corresponding signatures if their template-parameter-lists have the same length, corresponding template-parameters are equivalent, they have equivalent non-object-parameter-type-lists and return types (if any), and, if both are non-static members, they have corresponding object parameters.

2. Change in 6.4.1 [basic.scope.scope] paragraph 4 as follows:

- ...
- - ...
 - both declare function templates with **corresponding signatures and** equivalent ~~non-object-parameter-type-lists, return types (if any), template-heads, and trailing requires-clauses (if any), and, if both are non-static members, they have corresponding object parameters.~~

3. Change in 13.7.7.2 [temp.over.link] paragraph 7 as follows:

If the validity or meaning of the program depends on whether two constructs are equivalent, and they are functionally equivalent but not equivalent, the program is ill-formed, no diagnostic required. **Furthermore, if two function templates with corresponding signatures do not correspond, but accept and are satisfied by the same set of template argument lists, the program is ill-formed, no diagnostic required.**

Proposed resolution (approved by CWG 2022-09-09):

1. Append to 6.4.1 [basic.scope.scope] paragraph 3 as follows:

- ...
- the types of their object parameters are equivalent.

Two function templates have corresponding signatures if their template-parameter-lists have the same length, corresponding template-parameters are equivalent, they have equivalent non-object-parameter-type-lists and return types (if any), and, if both are non-static members, they have corresponding object parameters.

2. Change in 6.4.1 [basic.scope.scope] paragraph 4 as follows:

- ...
- - ...
 - both declare function templates with **corresponding signatures and** equivalent ~~non-object-parameter-type-lists, return types (if any), template-heads, and trailing requires-clauses (if any), and, if both are non-static members, they have corresponding object parameters.~~

3. Change in 13.7.7.2 [temp.over.link] paragraph 7 as follows:

If the validity or meaning of the program depends on whether two constructs are equivalent, and they are functionally equivalent but not equivalent, the program is ill-formed, no diagnostic required. **Furthermore, if two function templates that do not correspond**

- **have the same name,**
 - **have corresponding signatures (6.4.1 [basic.scope.scope]),**
 - **would declare the same entity (6.6 [basic.link]) considering them to correspond, and**
 - **accept and are satisfied by the same set of template argument lists,**
- the program is ill-formed, no diagnostic required.**

2604. Attributes for an explicit specialization

It is unclear whether an explicit template specialization "inherits" the attributes written on the primary template, or whether the specialization has to repeat the attributes. For example:

```
template <typename Ty>
[[noreturn]] void func(Ty);

template <>
void func<int>(int) {
    // Warning about returning from a noreturn function or not?
}
```

A similar question arises for attributes written on the parameters of the primary function template. For example:

```
template <typename Ty>
void func([[maybe_unused]] int i);

template <>
void func<int>(int i) {
    // i is not used, should it be warned on or not?
}
```

There is implementation divergence for the example.

Suggested resolution [SUPERSEDED]:

Change in 13.9.4 [temp.expl.spec] paragraph 13 as follows:

Any attributes applying to any part of the declaration of an explicit specialization of a function or variable template, as well as ~~Whether whether such~~ an explicit specialization of a function or variable template is inline, constexpr, or an immediate function, is determined by the explicit specialization and is independent of those properties of the template. **[Note: Attributes that would affect the association of the declaration of an explicit specialization with the declaration of the primary template need to match. -- end note]**

Proposed resolution (approved by CWG 2022-11-09):

Change 13.9.4 [temp.expl.spec] paragraph 13 as follows:

Whether an explicit specialization of a function or variable template is inline, constexpr, or an immediate function is determined by the explicit specialization and is independent of those properties of the template. **Similarly, attributes appearing in the declaration of a template have no effect on an explicit specialization of that template.** [Example 7:

```
template<class T> void f(T) { /* ... */ }
template<class T> inline T g(T) { /* ... */ }

template<> inline void f<>(int) { /* ... */ } // OK, inline
template<> int g<>(int) { /* ... */ }          // OK, not inline

template<typename> [[noreturn]] void h([[maybe_unused]] int i);
template<> void h<int>(int i) {
    // Implementations are expected not to warn that the function returns but can
    // warn about the unused parameter.
}
```

—end example]

2605. Implicit-lifetime aggregates

Section: 11.2 [class.prop] **Status:** ready **Submitter:** Davis Herring **Date:** 2022-06-27

Subclause 11.2 [class.prop] paragraph 9 specifies:

A class S is an *implicit-lifetime class* if

- it is an aggregate or
- it has at least one trivial eligible constructor and a trivial, non-deleted destructor.

However, an aggregate may have a non-deleted non-trivial destructor:

```
struct X {
    Y i;
    ~X();
};
```

This class is an aggregate, but destroying X itself (ignoring the subobjects) does not satisfy "destroying an instance of the type runs no code"; see P0593R6 "Implicit creation of objects for low-level object manipulation" section 3.1.

Additional notes (September, 2022):

From a thread starting [here](#): What if x had a deleted destructor (either explicitly or implicitly)?

CWG 2022-11-09:

A deleted destructor does not prevent an aggregate from being an implicit-lifetime class.

Proposed resolution (approved by CWG 2022-11-09):

Change in 11.2 [\[class.prop\]](#) paragraph 9 as follows:

A class S is an *implicit-lifetime class* if

- it is an aggregate **whose destructor is not user-provided** or
- it has at least one trivial eligible constructor and a trivial, non-deleted destructor.

2610. Indirect private base classes in aggregates

Section: 9.4.2 [\[dcl.init.aggr\]](#) **Status:** ready **Submitter:** Chris Bowler **Date:** 2022-07-21

The wording appears to prohibit aggregates from having indirect private base classes. That does not match existing practice and is an unnecessary restriction. For example:

```
#include <type_traits>

struct B1 {};
struct B2 : private B1 {};
struct S : public B2 {};

void f() {
    static_assert(std::is_aggregate<S>::value);
}
```

Proposed resolution (approved by CWG 2022-08-26):

Change in 9.4.2 [\[dcl.init.aggr\]](#) paragraph 1 as follows:

An aggregate is an array or a class (Clause 11 [\[class\]](#)) with

- no user-declared or inherited constructors (11.4.5 [\[class.ctor\]](#)),
- no private or protected direct non-static data members (11.8 [\[class.access\]](#)),
- **no private or protected direct base classes (11.8.3 [\[class.access.base\]](#)), and**
- no virtual functions (11.7.3 [\[class.virtual\]](#)) **or virtual base classes (11.7.2 [\[class.mi\]](#)), and**
- ~~no virtual, private, or protected base classes (11.7.2 [\[class.mi\]](#)).~~

2611. Missing parentheses in expansion of fold-expression could cause syntactic reinterpretation

Section: 13.7.4 [temp.variadic] **Status:** ready **Submitter:** Richard Smith **Date:** 2022-08-05

13.7.4 [temp.variadic] paragraph 10 expands a *fold-expression* (including its enclosing parentheses) to an unparenthesized expression. If interpreted literally, this could result in reassociation and misinterpretation of the expression. For example, given:

```
template<int ...N> int k = 2 * (... + N);
```

... k<1, 2, 3> is specified as expanding to `int k<1, 2, 3> = 2 * 1 + (2 + 3);` resulting in a value of 7 rather than the intended value of 12.

Further, there is implementation divergence for the following example:

```
#include <type_traits>
template<class ...TT>
void f(TT ...tt) {
    static_assert(std::is_same_v<decltype((tt, ...)), int&>);
}
template void f(int /*,int*/);
```

gcc and MSVC apply the general *expression* interpretation of `decltype`, whereas clang and icc apply the *identifier* special case.

Proposed resolution (approved by CWG 2022-08-26):

Change in 13.7.4 [temp.variadic] paragraph 10 as follows:

The instantiation of a *fold-expression* (7.5.6 [expr.prim.fold]) produces:

- $((E_1 \text{ op } E_2) \text{ op } \dots) \text{ op } E_N$ for a unary left fold,
- $E_1 \text{ op } (\dots \text{ op } (E_{N-1} \text{ op } E_N))$ for a unary right fold,
- $((E \text{ op } E_1) \text{ op } E_2) \text{ op } \dots \text{ op } E_N$ for a binary left fold, and
- $E_1 \text{ op } (\dots \text{ op } (E_{N-1} \text{ op } (E_N \text{ op } E)))$ for a binary right fold.

...

2612. Incorrect comment in example

Section: 9.4.1 [dcl.init.general] **Status:** ready **Submitter:** Jiang An **Date:** 2022-05-24

Subclause 9.4.1 [dcl.init.general] bullet 16.6.1 says:

[Example 2: `T x = T(T());` calls the `T` default constructor to initialize `x`. —end example]

This is incorrect; in some situations, the default constructor is not invoked (see 9.4.1 [dcl.init.general] paragraph 9).

Proposed resolution (approved by CWG 2022-08-26):

Change in 9.4.1 [dcl.init.general] bullet 16.6.1 as follows:

[Example 2: `T x = T(T());` ~~calls the `T` default constructor to initialize~~ **value-initializes** `x`. —end example]

2613. Incomplete definition of resumer

Section: 9.5.4 [dcl.fct.def.coroutine] **Status:** ready **Submitter:** Jim X **Date:** 2022-02-15

Subclause 9.5.4 [dcl.fct.def.coroutine] paragraph 8 specifies:

A suspended coroutine can be resumed to continue execution by invoking a resumption member function (17.12.4.6 [coroutine.handle.resumption]) of a coroutine handle (17.12.4 [coroutine.handle]) that refers to the coroutine. The function that invoked a resumption member function is called the *resumer*.

However, non-functions can also resume a coroutine, for example:

```
Task task() {
    std::cout << "in task\n";
    int r = co_await Line();
    std::cout << "resumed\n";
    co_return r;
}
auto r = task();
auto c = (r.coro_.resume(), 0); // #1
```

Proposed resolution (approved by CWG 2022-08-26):

Change in 9.5.4 [dcl.fct.def.coroutine] paragraph 8 as follows:

A suspended coroutine can be resumed to continue execution by invoking a resumption member function (17.12.4.6 [coroutine.handle.resumption]) of a coroutine handle (17.12.4 [coroutine.handle]) that refers to the coroutine. The **function evaluation** that invoked a resumption member function is called the *resumer*.

2614. Unspecified results for class member access

Section: 7.6.1.5 [expr.ref] **Status:** ready **Submitter:** Andrey Erokhin **Date:** 2021-10-27

Subclause 7.6.1.5 [expr.ref] paragraph 6 specifies:

If E2 is declared to have type “reference to T”, then E1.E2 is an lvalue; the type of E1.E2 is T. Otherwise, ...

This does not specify which object or function the resulting lvalue designates. A similar problem exists with member enumerators:

If E2 is a member enumerator and the type of E2 is T, the expression E1.E2 is a prvalue. The type of E1.E2 is T.

Proposed resolution (approved by CWG 2022-09-23):

1. Split and change in 7.6.1.5 [expr.ref] paragraph 6 as follows:

If E2 is declared to have type “reference to T”, then E1.E2 is an lvalue; **the of type of E1.E2 is T. If E2 is a static data member, E1.E2 designates the object or function to which the reference is bound, otherwise E1.E2 designates the object or function to which the corresponding reference member of E1 is bound.**

Otherwise, ...

2. Change in 7.6.1.5 [expr.ref] bullet 6.5 as follows:

If E2 is a member enumerator and the type of E2 is T, the expression E1.E2 is a prvalue. **The of type of E1.E2 is T whose value is the value of the enumerator.**

2616. Imprecise restrictions on break and continue

Section: Clause 8 [stmt.stmt] **Status:** ready **Submitter:** Jim X **Date:** 2022-08-24

Consider:

```
for (int i = 0; i < 10; ++i){
    auto f = [](){
        break; // #1
    };
}
```

Subclause 8.7.2 [[stmt.break](#)] paragraph 1 specifies:

The break statement shall occur only in an *iteration-statement* or a switch statement and causes termination of the smallest enclosing *iteration-statement* or switch statement; control passes to the statement following the terminated statement, if any.

Does the break at #1 "occur" in the for loop?

Proposed resolution (approved by CWG 2022-08-26):

1. Append to 8.1 [[stmt.pre](#)] paragraph 3 as follows:

... **A statement S1 is enclosed by a statement S2 if S2 encloses S1.**

The rules for conditions apply both...

2. Change in 8.2 [[stmt.label](#)] paragraph 2 as follows:

~~Case labels and default labels shall occur only in switch statements~~ **A labeled-statement whose label is a case or default label shall be enclosed by (8.1 [[stmt.pre](#)]) a switch statement (8.5.3 [[stmt.switch](#)]).**

3. Change in 8.7.2 [[stmt.break](#)] paragraph 1 as follows:

~~The~~ **A break statement shall be enclosed by (8.1 [[stmt.pre](#)]) an iteration-statement (8.6 [[stmt.iter](#)]) or a switch statement and (8.5.3 [[stmt.switch](#)]).** ~~The break statement~~ causes termination of the smallest **such** enclosing ~~iteration-statement or switch~~ statement; control passes to the statement following the terminated statement, if any.

4. Change in 8.7.3 [[stmt.cont](#)] paragraph 1 as follows:

~~The~~ **A continue statement shall be enclosed by (8.1 [[stmt.pre](#)]) an iteration-statement (8.6 [[stmt.iter](#)]) and.** ~~The continue statement~~ causes control to pass to the loop-continuation portion of the smallest **such** enclosing ~~iteration-statement~~ **statement**, that is, to the end of the loop. ...

2618. Substitution during deduction should exclude exception specifications

Section: 13.10.3.1 [[temp.deduct.general](#)] **Status:** ready **Submitter:** Christof Meerwald **Date:** 2021-11-27

Subclause 13.10.3.1 [[temp.deduct.general](#)] paragraph 7 specifies:

The substitution occurs in all types and expressions that are used in the function type and in template parameter declarations. The expressions include not only constant expressions such as those that appear in array bounds or as nontype template arguments but also general expressions (i.e., non-constant expressions) inside sizeof, decltype, and other contexts that allow non-constant expressions. The substitution proceeds in lexical order and stops when a condition that causes deduction to fail is encountered. If substitution into different declarations of the same function template would cause template instantiations to occur in a different order or not at all, the program is ill-formed; no diagnostic required.

[Note 4: The equivalent substitution in exception specifications is done only when the *noexcept-specifier* is instantiated, at which point a program is ill-formed if the substitution results in an invalid type or expression. —end note]

The note says that substitution into the *noexcept-specifier* occurs late, but the normative text does not support that, because the exception specification is part of the function type.

Subclause 13.10.3.1 [temp.deduct.general] paragraph 8 specifies:

Only invalid types and expressions in the immediate context of the function type, its template parameter types, and its *explicit-specifier* can result in a deduction failure.

However, paragraph 7 does not mention the *explicit-specifier* when describing the substitution.

Proposed resolution (approved by CWG 2022-09-09):

Change in 13.10.3.1 [temp.deduct.general] paragraph 7 as follows:

The deduction substitution loci are

- **the function type outside of the *noexcept-specifier*,**
- **the *explicit-specifier*, and**
- **the template parameter declarations.**

The substitution occurs in all types and expressions that are used in the ~~function type and in template parameter declarations~~ **deduction substitution loci**. ...

Change in 13.10.3.1 [temp.deduct.general] paragraph 8 as follows:

... ~~Only invalid~~ **Invalid** types and expressions **can result in a deduction failure only** in the immediate context of the ~~function type, its template parameter types, and its *explicit-specifier*~~ **deduction substitution loci** ~~can result in a deduction failure.~~

2619. Kind of initialization for a *designated-initializer-list*

Section: 9.4.2 [dcl.init.aggr] **Status:** ready **Submitter:** Jim X **Date:** 2022-07-13

Consider:

```
struct S {
    explicit S(int){}
};
struct A {
    S s;
};
struct B {
    union {
        S s;
    };
};
int main() {
    A a1 = {.s{0}};    // #1
    A a2{.s{0}};       // #2
    B b1 = {.s{0}};    // #3
    B b2{.s{0}};       // #4
}
```

Subclause 9.4.2 [dcl.init.aggr] bullet 4.2 specifies:

Otherwise, the element is copy-initialized from the corresponding *initializer-clause* or is initialized with the *brace-or-equal-initializer* of the corresponding *designated-initializer-clause*.

It is unclear what kind of initialization is performed for "is initialized". For example, one could imagine that the top-level kind of initialization is inherited. On the other hand, 9.4.1 [dcl.init.general] paragraph 14 specifies:

The initialization that occurs in the = form of a *brace-or-equal-initializer* or condition (8.5 [stmt.select]), as well as in argument passing, function return, throwing an exception (14.2 [except.throw]), handling an exception (14.4 [except.handle]), and aggregate member initialization (9.4.2 [dcl.init.aggr]), is called *copy-initialization*.

There is implementation divergence: gcc and icc reject the example; clang and MSVC accept.

Suggested resolution [SUPERSEDED]:

1. Change in 9.4.2 [[dcl.init.aggr](#)] bullet 4.1 as follows:

If the element is an anonymous union member and the initializer list is a brace-enclosed *designated-initializer-list*, the element is initialized by the ~~designated-initializer-list~~ **braced-init-list** { D }, where D is the *designated-initializer-clause* naming a member of the anonymous union member.

2. Change in 9.4.2 [[dcl.init.aggr](#)] bullet 4.2 as follows:

Otherwise, the element is copy-initialized from the corresponding *initializer-clause* or is ~~initialized~~ **copy-initialized or direct-initialized** with the *brace-or-equal-initializer* of the corresponding *designated-initializer-clause*, **according to the form of the brace-or-equal-initializer (9.4.1 [[dcl.init.general](#)])**. ...

CWG telecon 2022-09-09:

The examples #1 to #4 should all be valid, direct-initializing the s member.

Proposed resolution (approved by CWG 2022-09-23):

1. Change in 9.4.1 [[dcl.init.general](#)] paragraph 14 as follows:

The initialization that occurs in the = form of a *brace-or-equal-initializer* or condition (8.5 [[stmt.select](#)]), as well as in argument passing, function return, throwing an exception (14.2 [[except.throw](#)]), handling an exception (14.4 [[except.handle](#)]), and aggregate member initialization **other than by a designated-initializer-clause** (9.4.2 [[dcl.init.aggr](#)]), is called *copy-initialization*.

2. Change in 9.4.2 [[dcl.init.aggr](#)] bullet 4.1 as follows:

If the element is an anonymous union member and the initializer list is a brace-enclosed *designated-initializer-list*, the element is initialized by the ~~designated-initializer-list~~ **braced-init-list** { D }, where D is the *designated-initializer-clause* naming a member of the anonymous union member.

3. Change in 9.4.2 [[dcl.init.aggr](#)] bullet 4.2 as follows:

Otherwise, the element is copy-initialized from the corresponding *initializer-clause* or is initialized with the *brace-or-equal-initializer* of the corresponding *designated-initializer-clause*. If that initializer is of the form *assignment-expression* or = *assignment-expression* and a narrowing conversion (9.4.5 [[dcl.init.list](#)]) is required to convert the expression, the program is ill-formed. **[Note: If the initialization is by designated-initializer-clause, its form determines whether copy-initialization or direct-initialization is performed.-- end note]**

2620. Nonsensical disambiguation rule

Section: 9.3.3 [[dcl.ambig.res](#)] **Status:** ready **Submitter:** Krystian Stasiowski **Date:** 2019-04-18

Subclause 9.3.3 [[dcl.ambig.res](#)] paragraph 1 specifies:

The ambiguity arising from the similarity between a function-style cast and a declaration mentioned in 8.9 [[stmt.ambig](#)] can also occur in the context of a declaration. In that context, the choice is between a function declaration with a redundant set of parentheses around a parameter name and an object declaration with a function-style cast as the initializer. Just as for the ambiguities mentioned in 8.9 [[stmt.ambig](#)], the resolution is to consider any construct that could possibly be a declaration a declaration.

The specification correctly describes an ambiguity between a function declaration and an object declaration, but resolves the ambiguity to a "declaration", which does not offer any insight.

Proposed resolution (2022-09-09) [SUPERSEDED]:

Change in 9.3.3 [[dcl.ambig.res](#)] paragraph 1 as follows:

... Just as for the ambiguities mentioned in 8.9 [stmt.ambig], the **The** resolution is to consider any construct that could possibly be a **function** declaration a **function** declaration.

Proposed resolution (approved by CWG 2022-09-23):

Change in 9.3.3 [dcl.ambig.res] paragraph 1 as follows:

The ambiguity arising from the similarity between a function-style cast and a declaration mentioned in 8.9 [stmt.ambig] can also occur in the context of a declaration. In that context, the choice is between a **function declaration with a redundant set of parentheses around a parameter name and** an object declaration with a function-style cast as the initializer **and a declaration involving a function declarator with a redundant set of parentheses around a parameter name**. Just as for the ambiguities mentioned in 8.9 [stmt.ambig], the resolution is to consider any construct, **such as the potential parameter declaration**, that could possibly be a declaration **to be** a declaration.

2622. Compounding types from function and pointer-to-member types

Section: Clause Annex B [implimits] **Status:** ready **Submitter:** Jim X **Date:** 2022-09-02

Clause Annex B [implimits] bullet 2.3 specifies:

- Pointer (9.3.4.2 [dcl.ptr]), array (9.3.4.5 [dcl.array]), and function (9.3.4.6 [dcl.fct]) declarators (in any combination) modifying a class, arithmetic, or incomplete type in a declaration [256].

This omits function types as the to-be-modified type, and ignores pointer-to-member declarators.

Proposed resolution (approved by CWG 2022-09-23):

Change in Clause Annex B [implimits] bullet 2.3 as follows:

- Pointer (9.3.4.2 [dcl.ptr]), **pointer-to-member (9.3.4.4 [dcl.mptr])**, array (9.3.4.5 [dcl.array]), and function (9.3.4.6 [dcl.fct]) declarators (in any combination) modifying a **class, arithmetic, or incomplete** type in a declaration [256].

2624. Array delete expression with no array cookie

Section: 7.6.2.9 [expr.delete] **Status:** ready **Submitter:** Blacktea Hamburger **Date:** 2022-08-22

Consider:

```
char *p = static_cast<char*>(operator new[](2));
p = new (p) char[2]; // #1
delete[] p;          // #2
```

Subclause 7.6.2.8 [expr.new] paragraph 16 specifies:

... When a *new-expression* calls an allocation function and that allocation has not been extended, the *new-expression* passes the amount of space requested to the allocation function as the first argument of type `std::size_t`. That argument shall be no less than the size of the object being created; it may be greater than the size of the object being created only if the object is an array and the allocation function is not a non-allocating form (17.6.3.4 [new.delete.placement]).
...

Subclause 7.6.2.9 [expr.delete] paragraph 2 specifies:

... In an array delete expression, the value of the operand of `delete` may be a null pointer value or a pointer value that resulted from a previous array *new-expression*. [Footnote: ...] If not, the behavior is undefined.

The non-allocating form of the *new-expression* at #1 is constrained not to place an array cookie at the start of the array. Yet, the array delete appears to be expected to divine that fact.

Proposed resolution (approved by CWG 2022-10-07):

Change in 7.6.2.9 [\[expr.delete\]](#) paragraph 2 as follows:

... In an array delete expression, the value of the operand of delete may be a null pointer value or a pointer value that resulted from a previous array *new-expression* **whose allocation function was not a non-allocating form (17.6.3.4 [\[new.delete.placement\]](#))**. [Footnote: ...] If not, the behavior is undefined.

2625. Deletion of pointer to out-of-lifetime object

Section: 6.7.3 [\[basic.life\]](#) **Status:** ready **Submitter:** Blacktea Hamburger **Date:** 2022-08-27

Consider:

```
struct S {};  
  
int main() {  
    S* p = new S;  
    p->~S();  
    delete p;  
}
```

This code appears to be allowed per 6.7.3 [\[basic.life\]](#) bullet 6.1:

The program has undefined behavior if:

- the object will be or was of a class type with a non-trivial destructor and the pointer is used as the operand of a *delete-expression*,
- ...

However, this calls the (trivial) destructor on *p twice. Invoking a non-static member function of an out-of-lifetime object is generally undefined behavior per 6.7.3 [\[basic.life\]](#) bullet 6.2 and 6.7.3 [\[basic.life\]](#) bullet 7.2. The rules ought to be consistent.

Proposed resolution (approved by CWG 2022-10-07):

Change in 6.7.3 [\[basic.life\]](#) bullet 6.1 as follows:

The program has undefined behavior if:

- ~~the object will be or was of a class type with a non-trivial destructor and~~ the pointer is used as the operand of a *delete-expression*,
- ...

2626. Rephrase ones' complement using base-2 representation

Section: 7.6.2.2 [\[expr.unary.op\]](#) **Status:** ready **Submitter:** Jim X **Date:** 2022-09-10

Subclause 7.6.2.2 [\[expr.unary.op\]](#) paragraph 10 specifies:

The operand of ~ shall have integral or unscoped enumeration type; the result is the ones' complement of its operand. Integral promotions are performed. The type of the result is the type of the promoted operand. There is an ambiguity in the grammar...

This should be phrased in terms of the base-2 representation similar to bitwise-AND, instead of alluding to some bit representation by using the term "ones' complement".

Proposed resolution (approved by CWG 2022-10-07):

Subclause 7.6.2.2 [[expr.unary.op](#)] paragraph 10 specifies:

The operand of `~` shall have integral or unscoped enumeration type; the result is the ones' complement of its operand. Integral promotions are performed. The type of the result is the type of the promoted operand. **Given the coefficients x_i of the base-2 representation (6.8.2 [[basic.fundamental](#)]) of the promoted operand x , the coefficient r_i of the base-2 representation of the result r is 1 if x_i is 0, and 0 otherwise.** There is an ambiguity in the grammar...

2627. Bit-fields and narrowing conversions

Section: 9.4.5 [[dcl.init.list](#)] **Status:** ready **Submitter:** Tim Song **Date:** 2021-08-13

Consider:

```
struct C {
    long long i : 8;
};

void f() {
    C x{1}, y{2};
    x.i <=> y.i; // error: narrowing conversion required (7.6.8 [expr.spaceship] bullet 4.1)
}
```

The rules for narrowing conversions in 9.4.5 [[dcl.init.list](#)] paragraph 7 consider only the source type, even though integral promotions can change the type of a bit-field to a smaller integer type without loss of value range according to 7.3.7 [[conv.prom](#)] paragraph 5:

A prvalue for an integral bit-field (11.4.10 [[class.bit](#)]) can be converted to a prvalue of type `int` if `int` can represent all the values of the bit-field; otherwise, it can be converted to `unsigned int` if `unsigned int` can represent all the values of the bit-field. If the bit-field is larger yet, no integral promotion applies to it. If the bit-field has enumeration type, it is treated as any other value of that type for promotion purposes.

There is implementation divergence in the handling of this example.

Proposed resolution (approved by CWG 2022-10-07):

Change in 9.4.5 [[dcl.init.list](#)] bullet 7.4 as follows:

A narrowing conversion is an implicit conversion

- ...
- from an integer type or unscoped enumeration type to an integer type that cannot represent all the values of the original type, except where
 - **the source is a bit-field whose width w is less than that of its type (or, for an enumeration type, its underlying type) and the target type can represent all the values of a hypothetical extended integer type with width w and with the same signedness as the original type or**
 - the source is a constant expression whose value after integral promotions will fit into the target type, or
- ...

2629. Variables of floating-point type as switch conditions

Section: 8.5.3 [[stmt.switch](#)] **Status:** ready **Submitter:** Jim X **Date:** 2022-09-07

Consider:

```
switch(float v = 0) {
    case 0: ;
}
```

Subclause 8.1 [stmt.pre] paragraph 5 specifies:

... The value of a *condition* that is an initialized declaration in a switch statement is the value of the declared variable if it has integral or enumeration type, or of that variable implicitly converted to integral or enumeration type otherwise. ...

That appears to permit variables of floating-point type, whose value can be converted to integral type. In contrast, *expressions* of floating-point type are prohibited by 8.5.3 [stmt.switch] paragraph 2:

The condition shall be of integral type, enumeration type, or class type. ...

Possible resolution [SUPERSEDED]:

Change in 8.1 [stmt.pre] paragraph 5 as follows:

... The value of a *condition* that is an initialized declaration in a switch statement is the value of the declared variable if it has integral or enumeration type, or of that variable implicitly converted to integral or enumeration type **if it has class type**; otherwise **the program is ill-formed**. ...

CWG telecon 2022-10-21:

Rewording is needed.

Proposed resolution (approved by CWG 2022-11-09):

1. Change in 8.1 [stmt.pre] paragraph 5 as follows:

~~... The value of a *condition* that is an initialized declaration in a switch statement is the value of the declared variable if it has integral or enumeration type, or of that variable implicitly converted to integral or enumeration type otherwise. ...~~

2. Change in 8.5.3 [stmt.switch] paragraph 2 as follows:

The value of a *condition* that is an initialized declaration is the value of the declared variable, or the value of the expression otherwise. The **value of the** condition shall be of integral type, enumeration type, or class type. If of class type, the condition is contextually implicitly converted (7.3 [conv]) to an integral or enumeration type. If the (possibly converted) type is subject to integral promotions (7.3.7 [conv.prom]), the condition is converted to the promoted type. ...

2630. Syntactic specification of class completeness

Section: 11.4.1 [class.mem.general] **Status:** ready **Submitter:** Jim X **Date:** 2022-07-04

Consider:

```
// translation unit 1
export module A;
export class X {};

// translation unit 2
import A;
X x; // is X complete at this point?
```

Subclause 11.4.1 [class.mem.general] paragraph 8 specifies:

A class is considered a completely-defined object type (6.8.1 [basic.types.general]) (or complete type) at the closing } of the *class-specifier*. ...

The syntactic (even lexical) reference to the closing `}` does not address the question when a different translation unit regards a class as complete. However, it seems this provision is entirely redundant given 6.3 [\[basic.def.odr\]](#) paragraph 13:

A definition of a class shall be reachable in every context in which the class is used in a way that requires the class type to be complete.

The standard never asks the question: "Is class X complete?"; it always specifies "X shall be complete" (otherwise the program is ill-formed).

Possible resolution [\[SUPERSEDED\]](#):

Change in 11.4.1 [\[class.mem.general\]](#) paragraph 8 as follows:

~~A class is considered a completely-defined object type (6.8.1 [\[basic.types.general\]](#)) (or complete type) at the closing `}` of the *class-specifier*. The~~ A class is regarded as complete within its complete-class contexts; otherwise it is regarded as incomplete within its own class *member-specification*.

CWG telecon 2022-10-21:

Explicitly refer to reachable definitions.

Proposed resolution (approved by CWG 2022-11-09):

Change in 11.4.1 [\[class.mem.general\]](#) paragraph 8 as follows:

~~A class is considered a completely-defined object type (6.8.1 [\[basic.types.general\]](#)) (or complete type) at the closing `}` of the *class-specifier*. The~~ A class is regarded as complete **where its definition is reachable and** within its complete-class contexts; otherwise it is regarded as incomplete within its own class *member-specification*.

2635. Constrained structured bindings

Section: 9.1 [\[dcl.pre\]](#) **Status:** ready **Submitter:** Corentin Jabot **Date:** 2022-10-20

Consider:

```
template<class T> concept C = true;
C auto [x, y] = std::pair{1, 2}; // ok?
```

Subclause 9.1 [\[dcl.pre\]](#) paragraph 6 specifies:

A simple-declaration with an identifier-list is called a structured binding declaration (9.6 [\[dcl.struct.bind\]](#)). If the decl-specifier-seq contains any decl-specifier other than static, thread_local, auto (9.2.9.6 [\[dcl.spec.auto\]](#)), or cv-qualifier s, the program is ill-formed.

Use of the word "contains" leads to an interpretation that any *placeholder-type-specifier* (9.2.9.6.1 [\[dcl.spec.auto.general\]](#)), possibly including a *type-constraint*, is valid here, since a *placeholder-type-specifier* is a *decl-specifier* and "contains" `auto`.

However, paper P1141R2 (Yet another approach for constrained declarations), applied in November 2018, expressly excludes structured bindings from constrained `auto`:

Structured bindings do deduce `auto` in some cases; however, the `auto` is deduced from the whole (and not from the individual components). It is somewhat doubtful that applying the constraint to the whole, as opposed to (for example) applying separately to each component, is the correct semantic. Therefore, we propose to defer enabling the application of constraints to structured bindings to separate papers.

Notwithstanding, clang, gcc, and MSVC accept the example.

Proposed resolution (approved by CWG 2022-10-21):

Change in 9.1 [\[dcl.pre\]](#) paragraph 6 specifies:

A simple-declaration with an *identifier-list* is called a *structured binding declaration* (9.6 [dcl.struct.bind]). If the *decl-specifier-seq* contains any *decl-specifier* other than *static*, *thread_local*, *auto* (9.2.9.6 [dcl.spec.auto]), or *cv-qualifiers*, the program is ill-formed. Each *decl-specifier* in the *decl-specifier-seq* shall be *static*, *thread_local*, *auto* (9.2.9.6 [dcl.spec.auto]), or a *cv-qualifier*. [Example:

```
template<class T> concept C = true;
C auto [x, y] = std::pair{1, 2}; // error: constrained placeholder-type-specifier not permitted for structured bindings
-- end example ]
```

2641. Redundant specification of value category of literals

Section: 5.13 [lex.literal] **Status:** ready **Submitter:** Andrey Erokhin **Date:** 2022-11-07

Subclause 7.5.1 [expr.prim.literal] paragraph 1 specifies:

... A *string-literal* is an lvalue designating a corresponding string literal object (5.13.5 [lex.string]), a *user-defined-literal* has the same value category as the corresponding operator call expression described in 5.13.8 [lex.ext], and any other literal is a prvalue.

Yet, there is redundant specification in 5.13.2 [lex.icon] paragraph 3:

The type of an *integer-literal* is the first type in the list in Table 9 corresponding to its optional *integer-suffix* in which its value can be represented. An *integer-literal* is a prvalue.

And in 5.13.6 [lex.bool] paragraph 1:

The Boolean literals are the keywords *false* and *true*. Such literals are prvalues and have type `bool`.

And in 5.13.7 [lex.nullptr] paragraph 1:

The pointer literal is the keyword `nullptr`. It is a prvalue of type `std::nullptr_t`.

Proposed resolution (approved by CWG 2022-11-10):

1. Change in 5.13.1 [lex.literal.kinds] paragraph 1 as follows:

There are several kinds of literals. [Footnote: ...]

literal:
 integer-literal
 character-literal
 floating-point-literal
 string-literal
 boolean-literal
 pointer-literal
 user-defined-literal

[Note: When appearing as an *expression*, a literal has a type and a value category (7.5.1 [expr.prim.literal]). -- end note]

2. Change in 5.13.2 [lex.icon] paragraph 3 as follows:

The type of an *integer-literal* is the first type in the list in Table 9 corresponding to its optional *integer-suffix* in which its value can be represented. ~~An *integer-literal* is a prvalue.~~

3. Change in 5.13.6 [lex.bool] paragraph 1 as follows:

The Boolean literals are the keywords *false* and *true*. Such literals ~~are prvalues and~~ have type `bool`.

4. Change in 5.13.7 [lex.nullptr] paragraph 1 as follows:

The pointer literal is the keyword `nullptr`. It ~~is a prvalue of~~ **has** type `std::nullptr_t`.

